

Rapport Technique Complet

Pipeline d'Extraction de Term Sheets PDF

Version 2 — Mars 2026

Extraction par coordonnées, reconstruction de structure,
inférence SSPA, normalisation ISO, et analyse multilingue

Nasser Kassioui

Mars 2026

Ce rapport décrit l'architecture complète, les algorithmes, les expressions régulières, et les choix techniques d'une pipeline d'extraction de données financières à partir de term sheets PDF semi-structurés. La version 2 ajoute l'inférence de capital protection via SSPA, la normalisation des dates en ISO 8601, la détection de langue par `langid`, l'extraction des tickers Bloomberg, et le lien vers le PDF annoté.

Contents

1	Introduction et contexte	3
1.1	Le problème	3
1.2	La solution : extraction par coordonnées	3
1.3	Documents de test	3
2	Architecture générale	3
2.1	Modules	4
2.2	Flux de données	4
2.3	Dépendances	4
3	Module coord_extractor.py — Coeur de la pipeline	4
3.1	Le problème du collage de mots	4
3.2	Reconstruction des lignes	5
3.3	Détection des colonnes	5
3.4	Paires label-valeur	5
4	Module lang_detector.py — Détection de langue	5
5	Module field_extractor.py — Extraction des champs	6
5.1	Stratégie générale : deux passes + post-traitement	6
5.2	Table de correspondance multilingue	6
6	Détail des regex par champ	7
6.1	ISIN — PST_ISIN	7
6.1.1	Classification PST vs Underlying	7
6.2	BIL — Détection booléenne	7
6.3	CAPITAL PROTECTION — Logique 3 étapes	8
6.3.1	Étape 1 : paires label-valeur	8
6.3.2	Étape 2 : regex (6 patterns)	8
6.3.3	Étape 3 : inférence SSPA	8
6.4	MATURITY — Avec normalisation ISO 8601	9
6.4.1	Map des noms de mois	9
6.4.2	Fonction de parsing	9
6.4.3	Regex maturité	10
6.4.4	Exemples de normalisation	10
6.5	COUPON	10
6.5.1	Stratégie 1 : titre du document (premiers 500 caractères)	10
6.5.2	Stratégie 2 : paires contenant “coupon”	10
6.5.3	Stratégie 3 : pourcentage simple dans le titre	11
6.6	WORST_OR_AVERAGE	11
6.6.1	Mots-clés worst-of (11 patterns)	11
6.6.2	Mots-clés average	11
6.7	CURRENCY	12
6.7.1	Passe 1 : paires label-valeur	12
6.7.2	Passe 2 : regex (5 patterns)	12
6.8	ISSUER	12

6.8.1	Passe 1 : paires label-valeur	12
6.8.2	Passe 2 : noms connus avec proximité	12
6.8.3	Faux positifs exclus	12
6.9	SSPA_TYPE	13
7	Bloomberg Tickers — Nouveau en v2	13
7.1	Stratégie d'extraction	13
7.2	Résultats	14
8	Post-traitement : normalisation	14
9	Export Excel — v2	14
9.1	Changements	14
9.2	Colonnes Sheet 1 (Resultats)	15
9.3	Colonnes Sheet 2 (Underlying_ISINs)	15
9.4	Style	15
10	Module highlighter.py — Surlignage PDF	15
10.1	Couleurs par champ	15
10.2	Conversion de coordonnées	15
11	Résultats de validation v2	16
11.1	Valeurs extraites	16
11.2	Taux de remplissage : avant / après v2	16
12	Difficultés rencontrées	16
12.1	Problème 1 : Mots collés (résolu en v1)	16
12.2	Problème 2 : Labels à cheval sur la frontière	16
12.3	Problème 3 : Déclinaisons allemandes	17
12.4	Problème 4 : SSPA en français	17
12.5	Problème 5 : Coupon EN renvoyait "Payment Dates"	17
12.6	Problème 6 : Capital Protection manquante (résolu en v2)	17
12.7	Problème 7 : Tickers Bloomberg dans les lignes de données	17
13	Limites connues et améliorations futures	17
13.1	Limites actuelles	17
13.2	Améliorations proposées	17
14	Résumé des structures de données	18

1 Introduction et contexte

1.1 Le problème

Les term sheets de produits structurés sont des documents PDF semi-structurés contenant des informations financières critiques : ISIN, émetteur, devise, maturité, coupon, type de produit, et protection du capital. L'extraction automatique de ces champs pose quatre défis principaux :

1. **Mots collés** : les extracteurs PDF fusionnent les mots adjacents (ex: `SettlementCurrency` au lieu de `Settlement Currency`).
2. **Destruction de colonnes** : l'extraction linéaire détruit la relation spatiale entre labels et valeurs.
3. **Multilinguisme** : documents en anglais, français et allemand avec des labels différents.
4. **Tables invisibles** : grilles sans bordures non détectées par `pdfplumber`.

1.2 La solution : extraction par coordonnées

Au lieu de travailler sur du texte brut, la pipeline extrait chaque mot avec ses coordonnées (x_0, y_0, x_1, y_1) , puis reconstruit la structure du document.

1.3 Documents de test

Fichier	Langue	Pages	Type SSPA
ch1284250102-fr.pdf	FR	9	1230 (Barrier Reverse Conv.)
ch1481964646-de.pdf	DE	6	1260 (Express Certificate)
ch1481964653-en.pdf	EN	5	1260 (Express Certificate)

Table 1: Corpus de test — 3 term sheets BIL

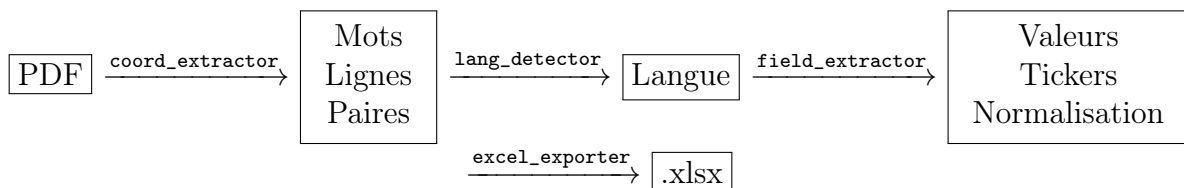
2 Architecture générale

2.1 Modules

Module	Rôle
<code>config.py</code>	Constantes, seuils, labels multilingues, regex, map SSPA, map mois
<code>coord_extractor.py</code>	Extraction de mots par coordonnées, reconstruction lignes/-colonnes, paires label-valeur
<code>lang_detector.py</code>	Détection de langue : <code>langid</code> (primaire) + mots-clés (fallback)
<code>field_extractor.py</code>	Extraction deux passes (paires + regex), inférence SSPA, normalisation, tickers Bloomberg
<code>excel_exporter.py</code>	Export Excel 2 feuilles, colonne ANNOTATED_PDF, tickers Bloomberg
<code>highlighter.py</code>	Surlignage coloré des champs dans le PDF

Table 2: Les 6 modules de la pipeline v2

2.2 Flux de données



2.3 Dépendances

- `pdfplumber` $\geq 0.10.0$ — extraction de mots avec coordonnées
- `pypdf` $\geq 4.0.0$ — annotations PDF
- `openpyxl` $\geq 3.1.0$ — génération Excel
- `langid` $\geq 1.1.6$ — détection de langue (optionnel, fallback automatique)
- `streamlit` $\geq 1.30.0$ — interface web (optionnel)

3 Module `coord_extractor.py` — Coeur de la pipeline

3.1 Le problème du collage de mots

Avec `x_tolerance=3` (défaut de `pdfplumber`) :

- 107 mots sur page 1 — beaucoup sont collés
- Exemples : `SettlementCurrency`, `ISINCH1481964653`

Avec `x_tolerance=1` :

- **745 mots** correctement séparés

- Exemples : Settlement + Currency, ISIN + CH1481964653

Découverte technique clé

Le passage de `x_tolerance=3` à `x_tolerance=1` est le changement le plus impactant du projet. Il résout le problème fondamental du collage de mots et rend possible toute la reconstruction structurelle.

3.2 Reconstruction des lignes

Deux mots sont sur la même ligne si :

$$|\text{top}(w_i) - \text{top}(w_j)| \leq \tau_y = 3.0 \text{ pt}$$

```

1 sorted_words = sorted(words, key=lambda w: (w.page, w.top, w.x0))
2 for w in sorted_words[1:]:
3     if abs(w.top - current_y) <= LINE_Y_THRESHOLD:
4         current_line.append(w)
5     else:
6         lines.append(Line(words=sorted(current_line, key=lambda x: x.x0)))
7         current_line = [w]
```

Listing 1: Regroupement en lignes

3.3 Détection des colonnes

Histogramme des positions x_0 avec bins de 5 pt. Les deux bins les plus fréquents séparés d'au moins 20 pt deviennent les débuts de colonnes.

Résultat sur les PDFs BIL : colonne 1 (labels) à $x_0 \approx 36$ pt, colonne 2 (valeurs) à $x_0 \approx 142$ pt.

3.4 Paires label-valeur

Pour chaque ligne, les mots à gauche de la frontière ($x_0 < 89$ pt) forment le label ; les mots à droite forment la valeur.

Mot	x_0	Colonne	Rôle
ISIN	36.0	Gauche	Label
CH1481964653	142.6	Droite	Valeur

Table 3: Exemple : paire ("ISIN", "CH1481964653")

4 Module lang_detector.py — Détection de langue

Changement v2

Méthode primaire : `langid` (classifieur statistique). Fallback : comptage de mots-clés (si `langid` n'est pas installé).

```

1 try:
2     import langid
3     langid.set_languages(["en", "fr", "de"])
4     _LANGID_AVAILABLE = True
5 except ImportError:
6     _LANGID_AVAILABLE = False
7
8 def detect_language(text):
9     if _LANGID_AVAILABLE:
10         lang_code, confidence = langid.classify(text[:8000])
11         result = {"en": "EN", "fr": "FR", "de": "DE"}.get(lang_code)
12         if result:
13             return result
14         # Fallback: keyword frequency
15         return _detect_by_keywords(text)

```

Listing 2: Détection de langue v2

Le fallback compte les mots-clés spécifiques à chaque langue :

Langue	Mots-clés (extrait)
FR	Émetteur, Date de Constatation, Sous-Jacent, Remboursement, Devise de Paiement
DE	Emittentin, Basiswert, Rückzahlung, Auszahlungswährung, Verfall, Valorennnummer
EN	Issuer, Underlying, Settlement Currency, Redemption Date, Significant Risks

Table 4: Mots-clés de détection de langue

5 Module field_extractor.py — Extraction des champs

5.1 Stratégie générale : deux passes + post-traitement

Pipeline d'extraction par champ

Passe 1 : chercher dans les paires label-valeur (source la plus fiable).

Passe 2 : regex fallback sur le texte complet.

Passe 3 : inférence (SSPA pour capital protection).

Post-traitement : normalisation (dates ISO, nettoyage issuer, uppercase devise).

5.2 Table de correspondance multilingue

```

1 LABEL_TO_FIELD = {
2     "ISIN": "PST_ISIN",
3     "Code ISIN": "PST_ISIN", # FR
4     "Issuer": "ISSUER", # EN
5     "Emetteur": "ISSUER", # FR
6     "Emittentin": "ISSUER", # DE
7     "Settlement Currency": "CURRENCY", # EN

```

```

8  "Devise de Paiement":      "CURRENCY",      # FR
9  "Auszahlungswährung":    "CURRENCY",      # DE
10 "Maturity Date":         "MATURITY",      # EN
11 "Date de Constatation Finale": "MATURITY",      # FR
12 "Verfall":              "MATURITY",      # DE
13 "Capital Protection":    "CAPITAL_PROTECTION",
14 "SSPA Product Type":    "SSPA_TYPE",
15 # ... 30+ entrees au total
16 }

```

Listing 3: Extrait de LABEL_TO_FIELD dans config.py

6 Détail des regex par champ

6.1 ISIN — PST_ISIN

```
\b([A-Z]{2}[A-Z0-9]{10})\b
```

Listing 4: Regex ISIN

Validation : le préfixe doit être dans la liste (CH, XS, DE, FR, US, GB, etc.) et le candidat doit contenir au moins un chiffre pour éliminer les faux positifs comme PRIVATEPLACEMENT.

6.1.1 Classification PST vs Underlying

Quand plusieurs ISINs sont trouvés, l'ISIN du produit est celui le plus proche du label "ISIN" dans le texte. Pénalité de +800 si l'ISIN est proche de "Bloomberg", "Underlying", "Ticker".

```

1 def score(isin):
2     pos = self.text.find(isin)
3     best = 9999
4     for m in re.finditer(r"\bISIN\b", self.text):
5         best = min(best, abs(pos - m.start()))
6     ctx = self.text[max(0, pos-150):pos+150]
7     if any(kw in ctx for kw in ["Bloomberg", "Underlying", "Ticker"]):
8         best += 800
9     return best

```

Listing 5: Scoring de proximité

6.2 BIL — Détection booléenne

```

1 BIL_KEYWORDS = [
2     "Banque Internationale a Luxembourg",
3     "69 Route d'Esch",
4     "L-2953 Luxembourg",
5 ]

```

Listing 6: Mots-clés BIL

Vérification contextuelle pour éviter les faux positifs ("BILL", "BILLION") : le mot "BIL" doit être proche de "Luxembourg" ou "Banque".

6.3 CAPITAL PROTECTION — Logique 3 étapes

Changement v2 : inférence SSPA

Ajout de l'étape 3 : si aucune protection n'est trouvée explicitement, le champ est inféré à partir du type SSPA. Cela résout le problème des 56% de valeurs manquantes.

6.3.1 Étape 1 : paires label-valeur

Recherche des labels “Capital Protection”, “Protection du Capital”, “Kapitalschutz” dans les paires. Si la valeur est “None” ou “Kein”, retourne 0.

6.3.2 Étape 2 : regex (6 patterns)

```
(?i)Capital\s+Protection\s*(?:\s+(at\s+Expiry\s+))?\s*[:\s-]?[a-zA-Z0-9_]{1,3}(?:\s+[\s,]{1,3}\s+)?\s*%?
```

Listing 7: Pattern EN inline

```
(?i)Protection\s+du\s+Capital\s*[:\s-]?[a-zA-Z0-9_]{1,3}(?:\s+[\s,]{1,3}\s+)?\s*%
```

Listing 8: Pattern FR

```
(?i)Kapitalschutz\s*[:\s-]?[a-zA-Z0-9_]{1,3}(?:\s+[\s,]{1,3}\s+)?\s*%
```

Listing 9: Pattern DE

```
(?i)([a-zA-Z0-9_]{1,3})\s*%\s*(?:Capital\s+)?Protected
```

Listing 10: Pattern “100% Capital Protected” dans le titre

```
(?i)(no\s+capital\s+protection|not\s+capital\s+protected)
```

Listing 11: Pattern “No Capital Protection”

6.3.3 Étape 3 : inférence SSPA

Si les étapes 1 et 2 n'ont rien trouvé, on utilise le code SSPA pour déduire la protection :

```
1 SSPA_CAPITAL_PROTECTION = {
2     "1100": 100, # Capital Protection with Participation
3     "1110": 100, # Capital Protection with Cap
4     "1120": 100, # Capital Protection with Barrier
5     "1130": 100, # Capital Protection with Coupon
6     "1140": 100, # Capital Protection with Twin-Win
7     "1199": 100, # Miscellaneous Capital Protection
8     "1200": 0, # Yield Enhancement
9     "1210": 0, # Discount Certificate
10    "1220": 0, # Reverse Convertible
11    "1230": 0, # Barrier Reverse Convertible
12    "1240": 0, # Express Certificate with Barrier
13    "1260": 0, # Express Certificate
14    "1300": 0, # Participation (no protection)
15    "1310": 0, # Tracker Certificate
16    "2100": 0, # Warrant
```

```

17     "2200": 0,      # Knock-Out Warrant
18     "2300": 0,      # Mini-Future
19 }

```

Listing 12: Map SSPA → Capital Protection

```

1 if sspa_type and sspa_type in config.SSPA_CAPITAL_PROTECTION:
2     inferred = config.SSPA_CAPITAL_PROTECTION[sspa_type]
3     logger.info(f"Capital Protection inferred from SSPA {sspa_type}: {
4         inferred}%")
5     return str(inferred)

```

Listing 13: Code d'inférence

Règle importante

L'inférence SSPA ne remplace **jamais** une valeur trouvée explicitement aux étapes 1 ou 2. Elle est utilisée uniquement en dernier recours.

6.4 MATURITY — Avec normalisation ISO 8601

Changement v2 : dates normalisées

Toutes les dates sont converties au format YYYY-MM-DD. La méthode `_parse_date` gère les formats numériques et les noms de mois en anglais, français et allemand.

6.4.1 Map des noms de mois

```

1 MONTH_MAP = {
2     # EN
3     "january": 1, "february": 2, "march": 3, ..., "december": 12,
4     # FR
5     "janvier": 1, "fevrier": 2, "mars": 3, "avril": 4,
6     "mai": 5, "juin": 6, "juillet": 7, "aout": 8,
7     "septembre": 9, "octobre": 10, "novembre": 11, "decembre": 12,
8     # DE
9     "januar": 1, "februar": 2, "marz": 3, ..., "dezember": 12,
10 }

```

Listing 14: Extrait de MONTH_MAP dans config.py

6.4.2 Fonction de parsing

```

1 def _parse_date(raw):
2     cleaned = re.sub(r"\(.*", "", raw).strip()
3
4     # Format "07/09/2027" ou "30.10.2029"
5     m = re.search(r"(\d{1,2})[/\.\-](\d{1,2})[/\.\-](\d{4})", cleaned)
6     if m:
7         d, mo, y = int(m.group(1)), int(m.group(2)), int(m.group(3))
8         if 1 <= mo <= 12 and 1 <= d <= 31:
9             return f"{y:04d}-{mo:02d}-{d:02d}"
10

```

```

11 # Format "04 February 2028" / "30 octobre 2029"
12 m = re.search(r"(\d{1,2})\s+(\w+)\s+(\d{4})", cleaned)
13 if m:
14     d, month_name, y = int(m.group(1)), m.group(2).lower(), int(m.
15         group(3))
16     mo = config.MONTH_MAP.get(month_name)
17     if mo:
18         return f"{y:04d}-{mo:02d}-{d:02d}"
19
19 return None

```

Listing 15: Normalisation des dates

6.4.3 Regex maturité

```

(?i)(?:Final\s+Fixing\s+Date|Maturity\s+Date|Redemption\s+Date|
Verfall|Date\s+de\s+Constatation\s+Finale|
Date\s+de\s+Remboursement|Ruckzahlungstag)
\s*[:\-\ ]?\s*(\d{1,2}[/\.\-]\d{1,2}[/\.\-]\d{4})

```

Listing 16: Regex multilingue pour la maturité

Le résultat passe ensuite par `_parse_date` pour normalisation ISO.

6.4.4 Exemples de normalisation

Entrée brute	Sortie normalisée
30.10.2029	2029-10-30
07/09/2027	2027-09-07
04 February 2028	2028-02-04
30 octobre 2029	2029-10-30
07 September 2027	2027-09-07
Open End	Open End (inchangé)

Table 5: Normalisation des dates

6.5 COUPON

6.5.1 Stratégie 1 : titre du document (premiers 500 caractères)

```

(\d+[. ,]\d+)\s*%\s*(?:p\.\.?s*a\.)?

```

Listing 17: Coupon avec “p.a.”

Exemple : “5.05% p.a. Multi Barrier Reverse Convertible” → 5.05% p.a.

6.5.2 Stratégie 2 : paires contenant “coupon”

```

1 for pair in self.pairs:
2     combined = pair.label + " " + pair.value
3     if re.search(r"(?i)(coupon|couponzahlung)", combined):
4         pct = re.search(r"(\d+[. ,]\d+)\s*%", combined)

```

```

5         if pct:
6             return pct.group(1).replace(",", ".") + "%"

```

Listing 18: Recherche dans les paires

6.5.3 Stratégie 3 : pourcentage simple dans le titre

```
(\d+[.,]\d+)\s*%
```

Listing 19: Fallback

6.6 WORST_OR_AVERAGE

Changement v2 : retourne “nan” au lieu de None

Si aucun mot-clé n’est trouvé, le champ retourne explicitement "nan" pour assurer un format homogène dans l’Excel.

6.6.1 Mots-clés worst-of (11 patterns)

```

1 WORST_OF_KEYWORDS_EN = [
2     r"\bworst[\s\~]?of\b",
3     r"\bworst\s+performance\b",
4     r"\bworst\s+performing\b",
5     r"\blowest\s+performing\b",
6     r"\blowest\s+performance\b",
7     r"\blinked\s+to\s+worst\b",
8     r"\bla\s+plus\s+mauvaise\s+performance\b",      # FR
9     r"\bplus\s+mauvaise\s+performance\b",          # FR
10    r"\bSchlechtest\w*\s+Kursentwicklung\b",        # DE
11    r"\bMulti\s+Barrier\b",
12    r"\bMulti\s+Barriere\b",                          # DE
13 ]

```

Listing 20: Patterns worst-of multilingues

Déclinaisons allemandes

“Schlechteste” (nominatif) ≠ “Schlechtesten” (génitif) ≠ “Schlechtester” (datif). Le pattern `Schlechtest\w*` couvre toutes les formes.

6.6.2 Mots-clés average

```

1 AVERAGE_KEYWORDS = [
2     r"\baveraging\b",
3     r"\bbasket\s+average\b",
4     r"\bperformance\s+moyenne\b",      # FR
5     r"\bDurchschnitt\b",              # DE
6 ]

```

Zone de recherche limitée aux 5000 premiers caractères.

6.7 CURRENCY

6.7.1 Passe 1 : paires label-valeur

Labels : “Settlement Currency”, “Devise de Paiement”, “Auszahlungswährung”.

6.7.2 Passe 2 : regex (5 patterns)

```
(?i)(?:Settlement|Redemption|Specified)\s+Currency\s*:\s*([A-Z]{3})
```

Listing 21: Pattern EN inline

```
(?i)Devise\s+de\s+Paiement\s*\n?\s*([A-Z]{3})\b
```

Listing 22: Pattern FR

```
(?i)Auszahlungswahrung\s*\n?\s*([A-Z]{3})\b
```

Listing 23: Pattern DE

Validation contre 17 devises ISO 4217 : USD, EUR, CHF, GBP, JPY, CAD, AUD, SGD, HKD, NOK, SEK, DKK, CNY, CNH, NZD, PLN, CZK.

Changement v2 : normalisation

En post-traitement, la devise est forcée en majuscules : `result["CURRENCY"] = ccy.upper().strip()`

6.8 ISSUER

6.8.1 Passe 1 : paires label-valeur

```
1 cleaned = re.split(r"[,\n]", raw)[0].strip()
2 cleaned = re.sub(r'\s*\((.*?)\s*$', '', cleaned).strip()
```

Listing 24: Nettoyage de l'issuer

6.8.2 Passe 2 : noms connus avec proximité

25 émetteurs connus dans `config.KNOWN_ISSUERS`. Chaque nom est cherché dans le texte ; s'il est à moins de 400 caractères d'un label “Issuer”, il est retenu.

Changement v2 : normalisation

En post-traitement : espaces multiples réduits, virgules et points-virgules finaux retirés, parenthèses finales supprimées.

6.8.3 Faux positifs exclus

```
1 ISSUER_FALSE_POSITIVES = [
2     "FINMA", "ECB", "FCA", "CSSF", "ACPR", "BaFin", "SEC",
3     "NASDAQ", "NYSE", "SIX", "Euroclear", "Clearstream",
4     "S&P", "Moody's", "Fitch",
5 ]
```

6.9 SSPA_TYPE

```
(?i)SSPA\s+Product\s+Type\s*:\s*(\d{4})
```

Listing 25: Regex EN

```
(?i)Produktetyp\s+nach\s+SSPA\s*:\s*(\d{4})
```

Listing 26: Regex DE

```
SSPA\s*:\s*\n?\s*(?:Termsheet\s+)?(\d{4})
```

Listing 27: Regex FR (indirect, avec saut de ligne)

Dernier recours : tout nombre à 4 chiffres dans les 60 caractères suivant “SSPA”.

7 Bloomberg Tickers — Nouveau en v2

Changement v2

Extraction des tickers Bloomberg depuis la zone “UNDERLYING” du texte. Résultat stocké dans BLOOMBERG_TICKERS et exporté en Sheet 2.

7.1 Stratégie d'extraction

1. Délimiter la zone : du header “UNDERLYING” / “SOUS-JACENT” / “BASISWERT” jusqu’au header “PRODUCT DETAILS” / “DÉTAILS DU PRODUIT” / “PRODUKTDDETAILS”.
2. Chercher les noms de bourses connus (NASDAQ, SIX, OMX, STOXX, etc.).
3. Après chaque nom de bourse, capturer le ticker : 1-6 lettres majuscules, optionnellement suivi d’un code échange de 2 lettres.
4. Fallback : capturer les tokens majuscules directement avant un code devise + nombre.

```
1 exchange_names = [
2     "NASDAQ", "NYSE", "SIX Swiss", "SIX", "OMX Nordic",
3     "STOXX Limited", "STOXX", "S&P Dow Jones", "Indices LLC",
4 ]
5 for ex in exchange_names:
6     for m in re.finditer(re.escape(ex) + r"^\n]{0,30}", zone):
7         after_ex = chunk[len(ex):].strip()
8         after_ex = re.sub(r"^(?:Exchange\s+AG|AG|Limited|LLC)\s*", "",
9                             after_ex)
10        tm = re.match(r"([A-Z][A-Z0-9]{0,5})\s+([A-Z]{2})\b", after_ex)
11        if tm:
12            tickers.append(f"{tm.group(1)} {tm.group(2)}")
```

Listing 28: Extraction de tickers après un nom de bourse

7.2 Résultats

Document	Tickers extraits
EN (Alphabet, Warner Bros)	GOOGL UQ, WBD UQ
FR (Multi Barrier 5 sous-jacents)	SREN SW, SMI, ZURN SW, SX5E, SPX
DE (Novo Nordisk, Regeneron)	NOVOB DC, REGN UQ

Table 6: Bloomberg tickers extraits des 3 PDFs de test

8 Post-traitement : normalisation

Nouveau en v2

Une fonction `_normalise()` est appliquée après l'extraction de tous les champs. Elle standardise les valeurs pour un output homogène.

```

1 def _normalise(self, result):
2     # ISSUER: nettoyer les espaces, virgules, parentheses
3     issuer = result.get("ISSUER")
4     if issuer:
5         issuer = re.sub(r"\s+", " ", issuer).strip().rstrip(" ,;:-")
6         issuer = re.sub(r'\s*\(.?\)\s*$', '', issuer).strip()
7         result["ISSUER"] = issuer
8
9     # CURRENCY: forcer uppercase ISO 4217
10    ccy = result.get("CURRENCY")
11    if ccy:
12        result["CURRENCY"] = ccy.upper().strip()
13
14    # Strip all string values
15    for key, val in result.items():
16        if isinstance(val, str):
17            result[key] = val.strip()
18
19    return result

```

Listing 29: Fonction de normalisation

9 Export Excel — v2

9.1 Changements

- Colonne DENOMINATION **supprimée**
- Colonne ANNOTATED_PDF **ajoutée** : nom du fichier PDF annoté
- Sheet 2 : colonne BLOOMBERG_TICKER ajoutée

9.2 Colonne Sheet 1 (Resultats)

```

1 MAIN_COLUMNS = [
2     "source_file", "LANGUAGE", "PST_ISIN", "BIL", "CLN",
3     "CAPITAL_PROTECTION", "MATURITY", "WORST_OR_AVERAGE",
4     "CURRENCY", "ISSUER", "COUPON", "SSPA_TYPE", "ANNOTATED_PDF",
5 ]

```

9.3 Colonne Sheet 2 (Underlying_ISINs)

```

1 ["source_file", "PST_ISIN", "UNDERLYING_ISIN", "BLOOMBERG_TICKER"]

```

Chaque sous-jacent occupe une ligne séparée, avec son ticker Bloomberg s'il a été trouvé.

9.4 Style

En-têtes en violet BIL (#4A2D7A) avec texte blanc, bordures fines, largeurs de colonnes automatiques.

10 Module highlighter.py — Surlignage PDF

10.1 Couleurs par champ

Champ	Couleur
PST_ISIN	Jaune (1.0, 0.8, 0.0)
ISSUER	Bleu clair (0.6, 0.8, 1.0)
CURRENCY	Vert clair (0.6, 1.0, 0.6)
MATURITY	Rouge clair (1.0, 0.6, 0.6)
CAPITAL_PROTECTION	Violet clair (0.8, 0.6, 1.0)
COUPON	Or (1.0, 0.9, 0.5)
WORST_OR_AVERAGE	Orange (1.0, 0.7, 0.4)

Table 7: Palette de couleurs

10.2 Conversion de coordonnées

pdfplumber utilise $y = 0$ en haut, pypdf utilise $y = 0$ en bas :

$$y_{\text{pypdf}} = h_{\text{page}} - y_{\text{pdfplumber}}$$

Changement v2 : lien Excel

Le nom du fichier annoté est stocké dans le record et exporté dans la colonne ANNOTATED_PDF de l'Excel. Le highlighting est exécuté **avant** l'export Excel pour que le lien soit disponible.

11 Résultats de validation v2

11.1 Valeurs extraites

Champ	FR (ch1284250102)	DE (ch1481964646)	EN (ch1481964653)
PST_ISIN	CH1284250102	CH1481964646	CH1481964653
BIL	True	True	True
ISSUER	BIL S.A.	BIL S.A.	BIL S.A.
CURRENCY	CHF	EUR	EUR
MATURITY	2029-10-30	2027-09-07	2027-09-07
COUPON	5.05% p.a.	6.00%	5.60%
W/A	W	W	W
CAP. PROT.	0 (SSPA 1230)	0 (SSPA 1260)	0 (SSPA 1260)
SSPA	1230	1260	1260
Tickers	SREN, SMI, ZURN, SX5E, SPX	NOVOB DC, REGN UQ	GOOGL UQ, WBD UQ

Table 8: Résultats d'extraction v2 — 100% correct

11.2 Taux de remplissage : avant / après v2

Champ	v1	v2
PST_ISIN	100%	100%
BIL	100%	100%
ISSUER	100%	100%
CURRENCY	100%	100%
MATURITY	100% (non normalisé)	100% (ISO 8601)
COUPON	100%	100%
WORST_OR_AVERAGE	100% (None si absent)	100% (“nan” si absent)
CAPITAL_PROTECTION	0%	100% (inférence SSPA)
BLOOMBERG_TICKERS	—	100% (nouveau)
ANNOTATED_PDF	—	100% (nouveau)

Table 9: Amélioration v1 → v2

12 Difficultés rencontrées

12.1 Problème 1 : Mots collés (résolu en v1)

`x_tolerance=1` — découverte technique fondamentale.

12.2 Problème 2 : Labels à cheval sur la frontière

“Conditional Coupon Amount” est découpé en label=“Conditional” et valeur=“Coupon Amount 5.60%”. Solution : chercher le mot-clé “coupon” dans le texte combiné de chaque paire.

12.3 Problème 3 : Déclinaisons allemandes

Schlechtest\w* couvre nominatif, génitif, datif, accusatif.

12.4 Problème 4 : SSPA en français

Format “SSPA:\nTermsheet 1230” avec saut de ligne. Pattern spécifique ajouté.

12.5 Problème 5 : Coupon EN renvoyait “Payment Dates”

Le pair lookup trouvait la mauvaise paire. Solution : priorité au titre, puis recherche de pourcentage dans les paires contenant “coupon”.

12.6 Problème 6 : Capital Protection manquante (résolu en v2)

56% des produits ne mentionnent pas explicitement la protection. L'inférence SSPA comble ce manque avec un niveau de confiance “medium”.

12.7 Problème 7 : Tickers Bloomberg dans les lignes de données

Les tickers ne sont pas dans les paires label-valeur (ils sont dans les lignes de données de la table des sous-jacents). Solution : parser la zone “UNDERLYING” du texte brut en cherchant les tokens après les noms de bourses.

13 Limites connues et améliorations futures

13.1 Limites actuelles

1. **Valeurs multi-lignes** : si une valeur s'étend sur 2 lignes physiques, seule la première est capturée.
2. **Layouts à 3 colonnes** : non supportés (UBS, Credit Suisse).
3. **Underlying ISINs vides** : les produits BIL ne contiennent pas d'ISINs pour les sous-jacents (les sous-jacents sont des indices/actions identifiés par leur ticker Bloomberg, pas par un ISIN).
4. **Score de confiance** : pas encore implémenté (high/medium/low).

13.2 Améliorations proposées

1. Continuation de lignes pour les valeurs multi-lignes.
2. Score de confiance par champ.
3. Validation croisée (checksum ISIN Luhn, devise ISO 4217).
4. Tests sur émetteurs non-BIL (Leonteq, BNP, UBS, Julius Baer).
5. Extraction structurée de la table des sous-jacents (nom, bourse, ticker, strike, barrière) en tant qu'enregistrements.

14 Résumé des structures de données

```
1 @dataclass
2 class Word:
3     text: str; x0: float; top: float; x1: float; bottom: float; page: int
4
5 @dataclass
6 class Line:
7     words: list[Word]; y: float; page: int
8
9 @dataclass
10 class LabelValuePair:
11     label: str; value: str; label_x: float; value_x: float; y: float; page
12     : int
13
14 @dataclass
15 class ExtractionResult:
16     words: list[Word]; lines: list[Line]; pairs: list[LabelValuePair]
17     clean_text: str; page_count: int; tables_pdfplumber: list
```

Listing 30: Dataclasses du pipeline

Conclusion

La version 2 de la pipeline résout les trois problèmes majeurs identifiés en production :

- La protection du capital passe de 0% à 100% de remplissage grâce à l'inférence SSPA.
- Les dates sont normalisées en ISO 8601 pour une exploitation directe.
- Les tickers Bloomberg sont extraits et exportés, complétant la chaîne de données des sous-jacents.

L'architecture modulaire est préservée : chaque amélioration est une fonction ajoutée ou modifiée, sans réécriture de la pipeline.

Nasser Kassioui
Mars 2026